

Detecting Repeated Milk Cartons in Clutter

Computer Vision Assignment 1

Jatin Purbia

B23CH1022

1 Object and Photographs

The object is the printed front face of an Amul Taaza milk carton. Its lettering, illustration, and lower graphics provide distinctive local features. The supplied original photographs are `template.jpeg` and `scene.jpeg`, both 1280×960 pixels.

The scene contains three cartons among a keyboard, book, green container, electronic device, and small bottle. The cartons have different rotations and are partly hidden; the right carton also reaches the image edge.

The template photograph includes considerable desk background. A manually specified rectangle

$$(x, y, w, h) = (592, 306, 241, 560)$$

isolates the carton before SIFT extraction. This is template preparation; no locations in the scene are annotated or supplied to the pipeline. Original photographs remain intact.



(a) Original template photograph



(b) Original cluttered scene

Figure 1: Original photographs used by the recognition pipeline.

2 Methods

2.1 Features and Matching

OpenCV SIFT produces position, size, orientation, and 128-dimensional descriptors. For each scene descriptor $\mathbf{s}$, the implementation computes squared Euclidean distance to every template descriptor $\mathbf{t}$ using

$$D^2(\mathbf{s}, \mathbf{t}) = \|\mathbf{s}\|^2 + \|\mathbf{t}\|^2 - 2\mathbf{s}^T \mathbf{t}.$$

Distances are clamped at zero for roundoff. Work is batched in blocks of 256 scene descriptors. The closest match is accepted if

$$d_1^2 < 0.78^2 d_2^2,$$

where d_1 and d_2 are the nearest and second-nearest descriptor distances. Ambiguous ties fail.

Scene-to-template matching preserves multiple copies of the same printed detail across the scene.

2.2 4D Generalized Hough Transform

Each match estimates relative scale and orientation:

$$s = \frac{\text{size}_{\text{scene}}}{\text{size}_{\text{template}}}, \quad \theta = \text{angle}_{\text{scene}} - \text{angle}_{\text{template}}.$$

Its predicted object center is

$$\hat{\mathbf{c}} = \mathbf{p}_{\text{scene}} + sR(\theta)(\mathbf{c}_{\text{template}} - \mathbf{p}_{\text{template}}),$$

where

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

Votes are accumulated in

$$(\hat{c}_x, \hat{c}_y, \log_2(s), \theta).$$

Each match votes in 16 neighboring cells, covering the lower and upper bin in all four dimensions. Angles wrap periodically. Identical candidate match sets are deduplicated and processed by decreasing support. Three votes create a candidate, not a detection.

2.3 Affine Verification

Custom RANSAC samples three correspondences without replacement. Repeated or collinear points and tiny triangles are rejected. The affine model is

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}.$$

The six affine coefficients are solved with centered NumPy least squares. Euclidean reprojection error is measured in original scene pixels:

$$e_i = \|A\mathbf{p}_{\text{template},i} + \mathbf{t} - \mathbf{p}_{\text{scene},i}\|_2.$$

Consistent points are counted once per template and scene location, reducing duplicate SIFT orientation support. Models with a reflection, collapsed area, unreasonable singular values, or excessive anisotropy are rejected.

The best model is repeatedly refitted to its inliers. RANSAC uses seed 7 and up to 1,200 trials, with adaptive stopping.

2.4 Greedy Extraction and Refinement

A Hough model seeds refinement across all remaining matches, allowing consistent features outside that Hough cell to join.

Acceptance requires:

- At least eight independent inliers.
- A convex hull covering at least 2.5% of the template rectangle.
- At least 25% of the predicted boundary area inside the image.

Only accepted geometric inliers are removed, including redundant orientations; unrelated matches inside a boundary remain. The process ends when no remaining cluster yields an acceptable model.

2.5 Boundaries and NMS

The four template-crop corners are transformed by each affine. Polygon intersection is implemented with Sutherland–Hodgman clipping; polygon area uses the shoelace formula.

Intersection over Union is

$$\text{IoU}(P, Q) = \frac{\text{area}(P \cap Q)}{\text{area}(P) + \text{area}(Q) - \text{area}(P \cap Q)}.$$

At IoU greater than 0.45, the less-supported detection is suppressed, breaking support ties with median reprojection error. This uses the oriented polygons rather than their axis-aligned enclosing rectangles.

All matching, voting, sampling, model scoring, refinement, convex hulls, polygon intersection, and suppression are implemented in `vision.py`. OpenCV is limited to SIFT, image I/O, color conversion, and rendering. No prohibited matcher, transformation estimator, homography, or clustering library is used.

3 Parameters

Parameter	Value	Reason
SIFT contrast threshold	0.025	Retain weaker printed details.
Lowe ratio	0.78	Preserve occluded-instance evidence while filtering ambiguous matches.
Position bin	70 scene pixels	Group moderately noisy center predictions.
$\log_2$ -scale bin	0.4	About a $1.32\times$ scale step.
Rotation bin	30°	12 circular bins.
Candidate support	3 votes	Minimum affine sample size; geometry must still verify.
RANSAC cap	1,200	Bounded robust estimation work.
Reprojection threshold	5 scene pixels	Tolerate localization error and mild non-affine effects.
Minimum independent inliers	8	Require more support than a minimal three-point fit.
Minimum hull coverage	2.5%	Reject extremely concentrated support.
Affine singular-value limits	0.15–3.0	Reject tiny or excessively enlarged models.
Maximum singular-value ratio	3.0	Reject excessive stretching or shear.
NMS IoU threshold	0.45	Suppress strongly overlapping duplicate detections.

Table 1: Configuration used for the reference run.

These are sample configuration choices, not thresholds calibrated on a separate validation set. All are recorded in `config.json` and `outputs/results.json`.

4 Naive Matching Versus Final Detections

The naive view shows **160 deterministically sampled matches out of 768** after the ratio test and before Hough/geometric verification. Lines connect template features to multiple scene locations, with some remaining mismatches. It cannot on its own identify clean instance boundaries.



Figure 2: Tentative matching overlay before Hough clustering and geometric verification.

The final view shows only three boundaries and object labels. Hidden edges are inferred from the fitted carton face. A boundary can therefore cross the book or another occluder. It is not a segmentation of just the visible pixels.



Figure 3: Final instance boundaries after geometric verification and output refinement.

5 Results and Limitations

The reference run used Python 3.12.4, OpenCV 4.12.0, and NumPy 1.26.4. It extracted **1,078 template keypoints**, **6,592 scene keypoints**, and **768 tentative matches**.

Three detections passed verification and three remained after NMS. NMS therefore removed **zero detections in this photograph**; the photograph does not demonstrate its duplicate-suppression behavior.

The separate `test_polygon_iou_and_nms` test supplies two overlapping predictions with 10 and 8 inliers plus a distant prediction with 9 inliers. It verifies that NMS removes the weaker overlapping prediction and keeps both distinct objects.

ID	Location/color	Inliers	Median (px)	RMSE (px)	Hull (%)
1	Center diagonal / green	179	0.68	0.86	51.5
2	Right diagonal / yellow	157	1.44	1.63	34.9
3	Left upright / blue	113	0.91	1.17	44.1

Table 2: Independent inlier counts, reprojection errors, and template support hull coverage.

Visual inspection finds **3 of 3 cartons detected, 0 missed cartons, 0 unrelated detections, and 0 duplicate boundaries** on this photograph.

This is a manual instance-count assessment on one supplied scene, not a benchmark or a measured boundary-IoU score. No ground-truth polygons were supplied. Reprojection errors describe fit to the accepted feature correspondences, not boundary accuracy.

5.1 Runtime and Stopping Behavior

The initial run took approximately **6.67 seconds**: 0.87 seconds for SIFT, 0.23 seconds for matching, and 5.57 seconds for geometry, excluding image output. Timing depends on hardware and numerical-library threading.

After three extractions, 248 matches remained; none of the 66 remaining candidate clusters passed verification. The detector stopped naturally without being told to find three objects.

5.2 Geometric Limitations

The object is three-dimensional, while the model describes its approximately planar printed front. Boundaries exclude visible side panels and approximate perspective effects; the rectangular crop also contains a small background margin.

The right predicted corner extends slightly outside the photograph ($x \approx 1295$ for an image width of 1280). Drawing is clipped by the image extent, while JSON preserves the full prediction.

5.3 Scale Variation and Generalization

The photographs provide strong clutter, rotation, and occlusion. Their actual scale variation between scene instances is modest: fitted area-equivalent scales relative to the template are roughly 0.78–0.80.

Consequently, this scene alone does not establish performance under large size differences. The synthetic three-instance test separately checks scales 0.7, 0.9, and 1.1 with different rotations.

More photographs are needed to evaluate larger scale changes, perspective, illumination changes, blur, or heavier occlusion. Low-texture objects may provide insufficient SIFT support, and strongly overlapping true instances can be suppressed by IoU NMS.

5.4 Automated Tests

Seven automated tests pass, including:

1. Brute-force matching agreement.
2. Empty and ambiguous inputs.
3. Affine degeneracy.
4. Recovery with 60% outliers.
5. Polygon IoU and NMS.
6. Angle wraparound.
7. Multiple-instance extraction.

They validate algorithmic behavior independently of the visually reviewed photograph.

Reproduction

Run the following commands from the submitted codebase root:

```
python -m pip install -r requirements.txt
python run_pipeline.py --config config.json
python -m unittest discover -s tests -v
```

The submission includes `assets/`, the Python source, `config.json`, `requirements.txt`, this report, and `outputs/`.

Per-detection transforms, original match indices, unclipped corners, and diagnostic extraction rounds are available in `outputs/results.json`.